

Sign language recognition

Project Proposal

IVU Practicals (710.241)

WS 2024/2025

A. Uthor

jules.barreaux@student.tugraz.at

Report	5/7
Evaluation	5/5
Plan / Diagram	5/5
Sci Work / Ref	2/3 / 17

January 15, 2025

Graz

Contents

1	Introduction	2
2	Database	2
3	Application Description	2
3.1	Different Approaches	2
3.2	LSTM	3
3.2.1	Data Preparation	3
3.2.2	Training the Model	3
3.3	Result	4
3.4	3D CNN	4
3.4.1	Selection of a New Training Model	4
3.4.2	Data Preparation	4
3.4.3	Data Augmentation	5
3.5	Model Training and Results	5
3.5.1	Result	5
4	To conclude	6
4.1	Updated Project Plan	6

1 Introduction

For this assignment, I chose to develop a project involving an application for sign language recognition based on American Sign Language (ASL). The objective was to recognize the signs for various terms, words, and expressions and display their translation in real time on a screen.

2 Database

To develop this application, the first step was to find a dataset of various ASL (American Sign Language) signs. I came across a dataset created by a Microsoft research team that includes a total of 1,000 different words. Each word is performed by multiple individuals, resulting in approximately 30 videos per word captured by different people. The dataset contains various files:

- **MSASL_train.json**: A JSON file with 16,054 training samples.
- **MSASL_test.json**: A JSON file with 4,172 testing samples.
- **MSASL_val.json**: A JSON file with 5,287 validation samples.

reference
(URL,
paper,
tech report?)
(Add [1])
here

Each line in the dataset files contains the following information:

- **url**: A URL link to the video.
- **start_time**: The starting point of the clip in the original video (in seconds).
- **end_time**: The ending point of the clip in the original video (in seconds).
- **label**: The class (an integer between 0 and 1000).
- **signer_id**: The ID of the signer.
- **box**: The bounding box of the signer in the form [y0, x0, y1, x1].
- **text**: The gloss for this clip corresponding to the *label*.
- **width**: The width of the original video.
- **height**: The height of the original video.
- **fps**: The frame rate (frames per second) of the original video.

This dataset provides the foundational information needed to train and validate the sign language recognition model.

With this data and the significant variations within it (different people, contrast, brightness, angles, etc.), we can train a prediction model with high accuracy and for diverse cases and contexts. ✓

3 Application Description

3.1 Different Approaches

To implement my application, I experimented with two different methods, each with its own data preprocessing system. The first method involved implementing a system based on an RNN, specifically an LSTM. As this method did not yield conclusive results, I then sought an alternative that could provide better outcomes and offer more guidance on how to process the data

reference

before passing it to a model.

I eventually came across an article written by the researchers who developed the MS-ASL dataset. In this article, they detailed the theory behind their implementation of a model for sign language recognition, the I3D model. For the second method, I used a 3D CNN, following several instructions outlined by the researchers in their article. Since this approach yielded significantly better results, I decided to stick with it moving forward.

Below, I will present the two methods I worked on.

3.2 LSTM

3.2.1 Data Preparation

For this method, I prepared the data to pass it to the LSTM model to predict the signs in real time. At that time, I didn't have experience with CNNs or RNNs, so during the second method, I realized I had made many mistakes in this approach.

I decided to use **MediaPipe**, a Python library, to detect the hands of users in the videos. MediaPipe provides the positions of the hands for each frame of a video, normalized between 0 and 1 relative to the size of the frame.

If only one hand is detected, an empty hand (with positions set to 0) is added for the frame. Similarly, if no hand is detected, null hand positions are added. This ensures that uniform-sized vectors can be created. Each hand has 21 landmarks, with positions defined by x, y, z . Thus, for each frame, there are $2 \times 21 \times 3$ values.

This process creates files associated with each video, containing a list of vectors representing the hand positions detected by MediaPipe for each frame. Resizing the frames was not necessary, as MediaPipe normalizes the values automatically. Additionally, increasing the data quantity by modifying frames with contrast or other filters was unnecessary because MediaPipe detects hand landmarks independently of such criteria. Only the hand positions matter.

After this step, I created sequences of uniform length from the extracted landmarks for each video, forming a vector that represents the data for each video. Additionally, I extracted only the x and y coordinates.

To create this vector, I selected frames based on a calculated interval, such that I picked one vector every n -th frame, where

$$n = \frac{\text{number of vectors to take from the video}}{\text{total number of vectors in the video}}$$

(integrate meth in reading flow)

This ensures that if there are too many vectors, I could skip some without losing the video's representative nature. Conversely, if the video is too short, some vectors were duplicated.

These video vectors were then used in the LSTM model.

3.2.2 Training the Model

To train the model, I used the LSTM model, which is known to be relatively effective at understanding relationships within sequences of data. This is what differentiates it from CNNs, which are not suited for this purpose. The LSTM model operates with a memory cell and a hidden state, allowing it to effectively take into account all elements of the video without forgetting the beginning, while creating relationships between the data sequences. A validation step is regularly applied to monitor whether the model starts simply memorizing the training data and is no longer truly learning.

I built this model using TensorFlow. The training data was passed to it along with the corresponding labels.

3.3 Result

After this step, I attempted to apply the model in real time, but the result was that all my classes had nearly the same detection probability, regardless of what I did on the screen.

After implementing Method 2, I identified several errors in my approach. The first issue was that I did not download a sufficiently diverse range of data for each class. The second mistake was using the training data as validation data during the training process. Finally, I should have performed a data augmentation step, such as applying random cropping to specific areas of the videos for each class, which would have significantly increased the amount of data available for MediaPipe. ✓

refer to Sec. 3.4 - you never called it
Method 1
Method 2
explicitly
before

All of these factors contributed to the poor results of this method. Moreover, the use of MediaPipe, along with setting hand positions to zero when a hand was not detected, was also likely a source of problems. ✓

3.4 3D CNN

3.4.1 Selection of a New Training Model

After searching for a solution to improve the LSTM method, I came across the article written by the creators of the dataset I was using [1]. In this article, the researchers tested different models and various data processing approaches, including the use of keypoints, which helped me understand some mistakes I had made previously. The use of keypoints was implemented with a model I was unfamiliar with, the HCN. They then used the LSTM and I3D models, directly applying images without relying on keypoints. ✓ Reference

According to their results, significantly better accuracy was achieved with images, particularly when using the I3D model. This is why I decided to switch not only my model but also the type of data I was using, opting to work directly with images.

Method	ASL100	ASL200	ASL500	ASL1000
Naive classifier	4.86	2.49	1.05	0.58
VGG+LSTM [1, 2]	33.42	21.21	5.86	-
HCN [1]	73.98	60.29	43.83	32.50
I3D [1]	95.16	93.79	89.80	81.08

Is this a screenshot of a table? That's a no go.

This "thing" is not mentioned in the text, what should the reader do with it? What do the numbers mean? (Acc., Error, runtime, complexity, ... ?)

3.4.2 Data Preparation

To prepare the data, this time I downloaded all the videos available for each class in the dataset, using a script that automated the extraction of the segments corresponding to the specific sign in each video. I also ensured that there was now a separate set of validation and final test data.

Once all the videos were downloaded, I created a script to split the videos into a fixed number of frames. Similar to the LSTM method, I used an interval of the same size to extract a set of frames that represent the entire video. This means skipping some frames if the video is too long or duplicating some frames uniformly if the video is too short.

3.4.3 Data Augmentation

To provide the model with more data, I added certain data augmentation steps as described in the article. First, when a sequence was created, it was resized to a resolution of $256 \times N$ or $N \times 256$. Then, a random 244×244 crop was performed on the image to create a new sequence of images.

For other variations, I centered the 244×244 window in the middle of the resized image. Additional augmentations included variations in contrast, brightness, and a final variation that horizontally flipped the original video. This process allowed each original video to produce four different sequences, effectively quadrupling the amount of training data. ✓

3.5 Model Training and Results

For the creation of my model, I used PyTorch, as recommended after the status meeting. I chose the `r3d_18` model from PyTorch, which consists of 18 layers and is pre-trained on the Kinetics-400 dataset. I added a dropout layer to the model to prevent overfitting, which occasionally caused poor results. ✓

The model expects a PyTorch tensor with the shape:

Batch size $\times$ Number of channels (RGB = 3) $\times$ Number of frames $\times$ Height $\times$ Width.

I used a small batch size (4) because my computer could not handle processing more videos simultaneously. The model captures spatial and temporal relationships and movements within the sequences. ✓

I performed multiple passes of the data through the model, with a validation step after each pass using randomly selected data. Additionally, for each training iteration, I did not use all four augmented sequences generated per video but randomly selected one of them. This not only prevented the model from overfitting by simply memorizing the data but also reduced the computational load on my machine, which has relatively limited capabilities for this kind of task.

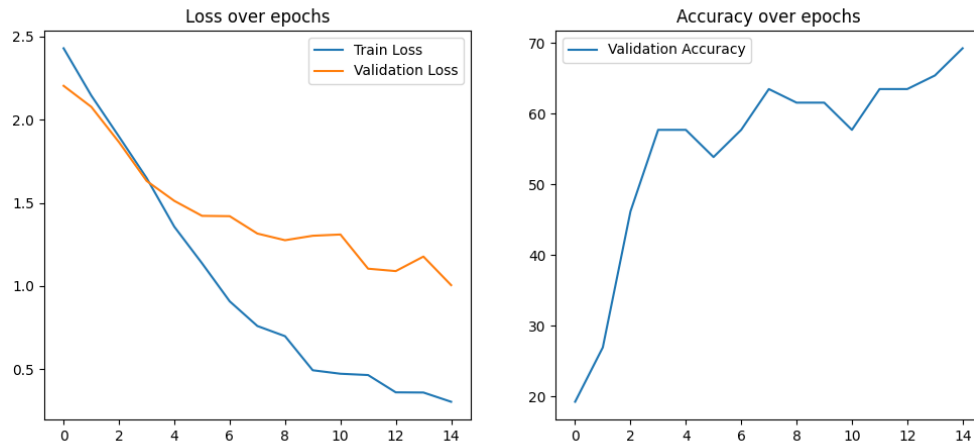
I typically performed between 15 and 20 passes of the data through the model during training. ✓

3.5.1 Result

Since my computer is not very powerful and my internet connection is extremely slow, I was only able to download videos for 10 classes out of the 1000 available in the dataset. Even with just 10 classes and data augmentation limited to $\times 4$, training the model takes approximately 10 hours. This is why I have not used more classes for now.

I configured the model with 15 epochs and 10 classes, and I obtained the following results: ✓

Always use labelled figures
and explicitly refer to
'Fig. X'.



Without explanation/definition of how "Acc" is defined in this setting, readers can only guess if this plot shows good performance (or could be better...)

We can see that the loss decreases drastically from the beginning to the end, although it starts to stagnate slightly towards the end. This indicates that the model is learning correctly to minimize errors on the training data. As for the validation loss, which represents the model's ability to generalize to unseen data, its progression becomes less significant. Judging by the curve, there seems to have been some room left before overfitting. ✓

Overall, the first graph indicates that the model is learning effectively and preparing itself to handle unseen data reasonably well.

The second graph shows the percentage of correct labels assigned to each video by the model during the training phase. Towards the end, the accuracy reaches just over 70%, which is relatively decent but significantly lower than the results reported by the researchers, who achieved 95% accuracy for 100 classes. ✓

I assume they performed more extensive data augmentation steps compared to mine, which likely accounts for this difference. Before applying augmentation, I barely achieved 40% accuracy for 10 different classes. ✓

I applied the trained model on 10 classes to the entire test dataset, meaning videos from different classes that the model had never seen before. On average, I achieved a 70% success rate. This is relatively good and should be a positive indicator for the next step, which will involve testing the model in real time. ✓

4 ~~To conclude~~ Conclusion

To conclude, I will need to download more data and perform additional data augmentation steps, such as varying brightness levels significantly, centering on different areas multiple times, and manipulating noise, colors, and so on. Finally, I will need to run the code on a larger dataset so that I can save a more refined model and test it in real time once I achieve a performance of at least 80% accuracy for, say, 50 different classes. ✓

4.1 Updated Project Plan

Having completely changed the implementation I initially had in mind for this project, the original plan no longer makes much sense. I had planned to learn how to use the learning model

and train it during the Christmas break, which I managed to achieve with the LSTM method before that period. ✓

I aimed to have a trained model ready by early January, which is, in a way, the current state as of today, although the model is not yet trained on enough data and classes. Finally, I had planned to test my model in real-time by January 17, which was the deadline, and that remains my current goal. ✓

References

- [1] Hamid Reza Vaezi Joze and Oscar Koller. MS-ASL: A Large-Scale Data Set and Benchmark for Understanding American Sign Language. *Microsoft Technical Report*, 2019.

Ⓢ + arXiv ID
? The report has
been released via
arXiv in 2018?